```
For every employee, calculate:

EmployeeID --> employee
Employee name --> employee
Department --> employee
Number of orders handled --> order

Number of orders whose total value exceeds ₹50,000 --> case when
-->
Total sales value of all orders handled --> order_items

Display only employees who have handled at least 5 orders. -->
having

Sort employees by total sales value from highest to lowest. -->
order by

o_id --> total_order_value

select
  e.EmployeeID,
  e.Name,
  e.Department,
  count(distinct o.OrderID) as orders_handled,
  count(
    distinct case
    when OrderTotals.order_value > 50000
    then o.OrderID
    end
  ) as HighValueOrders,
```

```
  sum(OrderTotals.order_value) as total_sales
from `e1.employees` as e
inner join `e1.orders` as o
on e.EmployeeID = o.EmployeeID
inner join (select
              OrderID,
              sum(Total) as order_value
            from `e1.order_items`
            group by OrderID) as OrderTotals
  on o.OrderID = OrderTotals.OrderID

  group by
  e.EmployeeID,
  e.Name,
  e.Department

having orders_handled >= 5
order by total_sales desc
```

For every employee, calculate:

EmployeeID --> employee
Employee name --> employee
Department --> employee
Number of orders handled --> orders

```
Number of different customers served --> orders

Display only employees who have handled more than 5 orders. -->
having

Sort employees by number of orders handled from highest to
lowest. --> order by



select
  e.EmployeeID,
  e.Name,
  e.Department,
  count(distinct o.OrderID) as total_orders,
  count(distinct o.CustomerID) as different_customers
from `e1.orders` as o
inner join `e1.employees` as e
on o.EmployeeID = e.EmployeeID
group by
  e.EmployeeID,
  e.Name,
  e.Department
having total_orders > 5
order by total_orders desc
```

```
For every city, calculate:

Number of customers --> customers
Number of orders --> orders
Total sales value --> order_items
Average order-item value --> order_items

Display only cities where the total sales value is greater than
₹3,00,000. --> having

Sort cities by total sales in descending order. --> order by
```

```sql
select
  c.City,
  count(distinct c.CustomerID) as no_of_customers,
  count(distinct o.OrderID) as no_of_orders,
  sum(oi.Total) as total_sales,
  round(avg(oi.Total),2) as avg_item_value
from `e1.customers` as c
inner join `e1.orders` as o
on c.CustomerID = o.CustomerID
inner join `e1.order_items` as oi
on o.OrderID = oi.OrderID
group by c.City
having total_sales> 300000
order by total_sales desc
```

```
For every product that has been sold, calculate:

ProductID --> product
ProductName --> product
Number of orders containing the product --> order_items
Total quantity sold --> order_items
Average selling price --> order_items

Display only products whose average selling price is greater
than ₹15,000. --> having

Show the highest average selling price first. --> order by
```

```sql
select
  p.ProductID,
  p.ProductName,
  count(distinct oi.OrderID) as no_of_orders,
  sum(oi.Quantity) as total_qty,
  avg(oi.SellingPrice) as avg_selling_price
from `e1.products` as p
inner join `e1.order_items` as oi
on p.ProductID = oi.ProductID
group by
  p.ProductID,
  p.ProductName
having avg_selling_price > 15000
```

```
order by avg_selling_price desc
```

For every category, calculate:

CategoryName --> category
Number of products --> product
Minimum MRP --> product
Maximum MRP --> product
Difference between maximum and minimum MRP --> product

Display only categories where the difference between the maximum and minimum MRP is greater than ₹10,000.

```
select
  c.CategoryName,
  count(distinct p.ProductID) as no_of_products,
  min(p.MRP) as min_mrp,
  max(p.MRP) as max_mrp,
  max(p.MRP) - min(p.MRP) as mrp_diff
from `e1.categories` as c
inner join `e1.products` as p
on c.CategoryID = p.CategoryID
group by  c.CategoryName
having mrp_diff > 100
```

```
For each payment method, calculate:

Number of successful transactions
Total successful payment amount
Average successful transaction amount
Maximum successful transaction amount

Display only payment methods having more than 10 successful
transactions.

Sort by average transaction amount from highest to lowest.


select *
from `e1.payments`
limit 5

select
  Method,
  count(*) as succ_trans,
  sum(Amount) as succ_amt,
  round(avg(Amount),2) as avg_amt,
  max(Amount) as max_amt
from `e1.payments`
```

```
where Status = "Success"
group by Method
having succ_trans > 10
order by avg_amt desc




Find the top 15 products based on total quantity sold.

Display:

ProductID --> product
ProductName --> product
CategoryName --> category
Total quantity sold --> order_items
Total sales value --> order_items

Sort by quantity sold from highest to lowest. --> order by



select
  p.ProductID,
  p.ProductName,
  c.CategoryName,
  sum(oi.Quantity) as total_qty,
  sum(oi.Total) as total_sales
from `e1.products` as p
inner join `e1.categories` as c
```

```
on p.CategoryID = c.CategoryID
inner join `e1.order_items` as oi
on p.ProductID = oi.ProductID
group by
  p.ProductID,
  p.ProductName,
  c.CategoryName
order by total_qty desc
limit 15




For every warehouse, calculate:

WarehouseID
Warehouse name
Number of inventory records
Total stock
Average stock
Highest stock in a single inventory record

Display only warehouses where the average stock per inventory
record is greater than 100.

Sort by total stock descending.
```

Find products that:

Exist in the product catalogue
Have inventory records
Have never appeared in any order item

Display:

ProductID
ProductName
SupplierID
Number of inventory records
Total stock

Sort by total stock from highest to lowest.

For each supplier, calculate:

SupplierName
Number of products
Average MRP
Highest MRP
Lowest MRP

Classify each supplier as:

Premium Supplier if average MRP >= ₹30,000
Mid-Range Supplier if average MRP >= ₹15,000
Budget Supplier otherwise

Display the suppliers with the highest average MRP first.

Find customers who have purchased products from at least 3 different categories.

```
Display:

CustomerID
Customer name
Number of different categories purchased
Total quantity purchased
Total amount spent

Sort by number of different categories purchased from highest to
lowest.
```